perplexity

A-prompt.md

The main defects are **semantic**: `Host` can certify a host against a description and observer that conceal its behavior; certification can admit a law despite a current counterexample; and transport accepts an alleged identity without evidence connecting the implementations. Below I review the six files separately, using your numbered categories. Quotes identify the relevant lines; findings are from the supplied snapshot, not from running its interpreter.[1]

`world.lth`

1. Laws that are wrong

```
match agreement ... { (w1, holds) -> holds }
```

Both `HostAgreesPrint` and `HostAgreesReadFile` discard the returned token. Likewise:

```
match o.observe w1 places { (w2, second) -> first == second }
```

`HostObserveChangesNothing` discards `w2`. These definitions do not implement the stated protocol in which "the token after is the runner's, which goes on with it." Under the supplied exactly-once rule, the final token must have a consumer. Moreover, these Bool-returning definitions execute world actions, contrary to the distinction between effect-free evaluation and execution. Extracting `holds` is not enough to return the token to the runner.[1]

2. Laws that do not determine what they claim

```
HostObserveChangesNothing ... first == second
```

Repeated equal observations do not establish that observation changes nothing. Countermodel: observing overwrites a file with a fixed value, then reports that value. The second observation reports the same state. The law passes although the first observation changed the world. A constant observer concealing all changes also passes.[1]

```
printDescribed : String -> Described S Unit world
```

Descriptions are supplied by the host model, with no law connecting them to `Console`. Countermodel: take a singleton representation `S`, a constant `world.read`, identity descriptions with `places := nil`, and an observer returning the singleton. Let `print` do nothing. Every `Host` agreement succeeds, and the descriptions satisfy `Described`, but `print` never prints. Thus `Host` does not establish the console behavior claimed by §4.[1]

```
o.observe w0 d.places
```

Agreement observes only declared places. A host that prints correctly but also corrupts an undeclared file passes if the observer reports only the declared projection. `ChangesOnlyDeclared` constrains the pure description, not the actual host action. There is no host-side frame law excluding undeclared changes. [1]

```
RecordCrossesByField ... b.callField [(l, f)] l xs == b.callBack f xs
```

Only singleton records are constrained. Countermodel: implement singleton records correctly, but return `(nil, 0)` for every record with two or more fields. All three `Crossing` laws pass; general records of functions do not cross faithfully. [1]

```
Externs : { verdictOf ... applied ... identity ... }
```

No operation or law connects the verdict to the declared type, its laws, the bytes actually loaded, or the order of validation and application. Countermodel: always return `Confirmed`, always apply, and use `h` as the identity; for provisional routes return `Assumes`. All `Externs` laws hold without any validation. [1]

3. Roles, L2.7

No local `data` declaration needs a constructor-role finding. `Route` is an alias, and `Verdict` is imported; its constructor laws are outside this review. [1]

5. Against the specification or decisions

```
e.applied r h == (r.provisional or settled (e.verdictOf r h))
```

This authorizes provisional application unconditionally. Chapter 10.S10 retains a provisional exception, but 6.S14 and D-2026-10-12-y refuse a demanded cone containing a provisional name. The file does not distinguish a provisional application outside certification from an application in a strict certified run. This is an unresolved interface contradiction, not a reason to reopen strictness. [1]

6. Implication over a type

```
not d.own or ...
```

`OwnIsIdempotent` should quantify over descriptions with `own == true`. Sampling arbitrary descriptions spends the non-own cases establishing nothing about idempotence. Similarly, `ProvisionalIsAssumed` should quantify over provisional routes. [1]

The exclusions in `ChangesOnlyDeclared` and `WriteChangesOnlyItsPlace` are different: their intended domains are undeclared places and distinct places, respectively. Quantifying over those subtypes makes the obligation explicit; these are not the most severe rare-premise problems here. [1]

7. What I would change first

Separate the execution protocol from its Boolean check: the protocol must return the successor token, the action's original result, and the recorded comparison. Keep the law predicate over that evidence effect-free. This removes the immediate token-loss problem and makes it possible for a checked `readFile` to deliver its result to the program.[1]

8. Cutoffs

No literal policy cutoff found. `0`, `1`, newline text, and place names are not tolerance or evidence thresholds. The more important omission is that `settled` has no demand-supplied acceptance criterion: `Supported` is accepted categorically.[1]

9. Certifying the host: missing laws

The missing obligations are:

- observation faithfully represents the requested places;
- observation preserves the world, rather than merely repeating its answer;
- each host description implements the intended operation—here, the relevant `Console` behavior;
- the actual action preserves undeclared places;
- the execution record binds observations, result, arguments, description, and host-code identity to the same execution.

`agreement` alone compares two model-supplied accounts; it does not establish that either account faithfully describes the world.[1]

`world_model.lut`

4. The model against its theory

```
    console.readFile := fn p st { (st, lookupFile st.files p) }
```

The pure `console` definitions satisfy the four displayed `Console` laws: printing appends a line and preserves other reads; reading preserves the state; reading a non-stdout place agrees with `theWorld.read`. The proposed stdout exclusion specifically avoids the apparent mismatch at `"stdout"`. I found no concrete pure-law violation here.[1]

The two `Described` values also satisfy their displayed laws: both are non-own actions; printing changes only `"stdout"` in the representation, and reading changes no represented state.[1]

```
    observe := extern "python:luthier.host.observe"
```

Whether `host` satisfies its laws cannot be established from this file: none of the three Python implementations is supplied.[1]

5. Against the specification

```
    the token comes first and goes back unchanged
```

As written, this comment conflicts with 10.S5's `act` law, "gives a new token each time." The actual implementation is not present, so this is a contradiction in the documented model, not a verified runtime violation.[1]

7. What I would change first

Make the relation between `host` and `console` an obligation of the host theory, rather than relying on this model's choice of description fields. The supplied descriptions are appropriate, but `Host` also admits inappropriate replacements.[1]

8–9. Cutoffs and host certification

No constant cutoff found. Host certification remains conditional on faithful, non-mutating observation and on host-side preservation of undeclared places; the extern declarations themselves establish neither.[1]

`certification.lth`

1. Laws that are wrong

```
    HashSeesTheHost ... hosts == c.hosts or not (... == h.hash c)
```

Together with the analogous code and interpreter laws, this requires collision-free hashing over unrestricted inputs. An intended fixed-width digest cannot satisfy that universal requirement: the input domain contains more distinct values than its finite output range. The declaration `Hash = String` does not itself force finite-width output, so the abstract theory is not thereby unsatisfiable; it excludes any fixed-width digest model.[1]

2. Laws that do not determine what they claim

```
    WeakestAcrossBindings ... c.entries == nil ...
```

```
    StrongestUnderOneBinding ... c.entries == nil ...
```

These determine grades for exactly two qualifying passes, but not for one pass or arbitrary larger certificates.

Countermodel: use the reference model for every other operation. For `grade`, discard irrelevant and not-run entries; return `nothing` on any current failure; use the specified aggregation only when there are exactly two current passes; return `nothing` otherwise. This satisfies the displayed grade laws, including both two-pass laws, yet a single proved pass produces no grade.[1]

```
    RefusalSaysWhy ... if anyServes k.serves d es then nil
```

A serving pass overrides a counterexample elsewhere in the same current law/cone. Concrete case: put a `Failed` entry and a covering `Passed` entry at the current hash, with different seeds so the store's determinism constraint permits both. `AFailureRefutes` requires the certificate grade to be `nothing`, but `RefusalSaysWhy` requires no refusal whenever the pass serves. Certification and demand acceptance disagree.[1]

```
Standing = { law : String, on : List ParameterReach, grade : Grade }

transport : Standing -> Standing -> String -> Standing
```

An arbitrary standing can be supplied as `i`, and an arbitrary target name as `a`. There is no identity proposition, pair of endpoints, cone identity, or certificate showing that `i` establishes `A ≡ B`. Countermodel: supply unrelated proved standings and request transport to any target. The resulting weaker grade and intersected reach satisfy every transport law, despite no evidence connecting the implementations.[1]

```
Entry = { ... evidence : Evidence, grade : Maybe Grade, ... }
```

The laws require a grade when an entry passed, but do not derive that grade from its evidence. A passed entry with assumed evidence and `just Proved` is not excluded by the displayed store predicate or certification laws. Likewise, no law restricts a `Cases` entry's reach to the cases actually checked. A forged `Cases` pass with `Support` can serve values never checked.[1]

3. Roles, L2.7

```
Value ... Number ... Printed ...
```

`Number` and `Printed` have operational roles in `ReachesAsSets`: intervals admit numeric values and reject printed values; member reaches use equality. Those are more than constructor enumeration. `Reach`, `Outcome`, and `State` also have substantive roles in containment, grading, serving, and refusal laws. I would not flag these constructors as role-free.[1]

The missing connection between `Failed` and demand refusal is nevertheless a defective role interaction: it refutes the certificate grade without necessarily preventing acceptance.[1]

5. Against the specification or decisions

```
record ... { just f -> st ; nothing -> cons e st }
```

The comment explicitly says `--deep` "rechecks and adds nothing." That contradicts 6.S8 and L10.4's requirement that a new check issue a new frozen entry, with both kept. Idempotently writing the same check result is not the same operation as performing a new check and suppressing its evidence.[1]

```
transport ... Standing -> Standing -> String -> Standing
```

Besides lacking identity evidence, transport has no distribution compatibility obligation. It therefore accepts sampled links from different distributions without the relating law required by 6.S12 and L10.8. Deferring cost types under D-2026-10-14-c does not supply that missing Boolean obligation.[1]

```
Entry ... grade : Maybe Grade
```

The older cost-based design can legitimately be deferred under D-2026-10-14-c. But that decision does not authorize treating an unconstrained stored grade as evidence, or dropping the distribution dependence required by L9.6. Those are evidence-integrity omissions, not merely deferred cost algebra. [1]

6. Implication over a type

The most serious instances are:

- `WeakestAcrossBindings` and `StrongestUnderOneBinding`: simultaneous matches on law, hash, outcomes, optional grades, bindings, and an empty certificate.
- `AFailureRefutes`: matched law/hash and a failed outcome.
- `ACoveringPassServes`: a matched current covering pass with a qualifying grade.

Use domains such as a current failed entry for this certificate, a pair of current graded passes with equal/distinct bindings, and an entry satisfying this demand's acceptance conditions. Independent random records will rarely reach those premises. [1]

7. What I would change first

Unify current-law validity and demand acceptance: a current counterexample must prevent the law from serving a demand under the file's own stated global-refutation policy. This fixes a direct path by which a refuted law enters `Certified`. [1]

8. Cutoffs

```
Demand = { ... accepts : Grade }
```

The grade cutoff is correctly supplied by the demand. No literal tolerance, rank floor, confidence level, or sample-count cutoff is hardcoded here. The problem is missing sampled-evidence parameters and obligations, not an identifiable numeric constant. [1]

9. Certifying the host

A successful host agreement needs a law connecting its actual recorded check to `Entry.evidence`, `outcome`, `grade`, and the exact case/domain checked. Otherwise a host can attach an unjustified grade or reach to a genuine successful comparison. [1]

`certification_model.lut`

4. The model against its theory

```
if anyServes serves d es then refuseLaws es n rest
```

The pass-plus-current-counterexample case above is reproduced by this model. It does not break the written `RefusalSaysWhy` law—it implements that defective law—but it violates the file's

claimed global-refutation semantics and chapter 6.S3. This distinction matters: the theory is too weak or wrongly connected, rather than the model disagreeing with that particular law.[1]

For the displayed pure operations, I found no unconditional concrete counterexample to a `Certifying` law. `grade` supplies the intended aggregation absent from the laws; `contains`, `serves`, and refusal mirror their specifications. A floating-point transport caveat appears in the final list below.[1]

```
hash := extern "python:luthier.certify.cone_hash"
```

Its compliance cannot be checked without the implementation. If it returns a fixed-width digest, it cannot satisfy the universal injectivity laws.[1]

7. What I would change first

Change refusal so that a current failure cannot be bypassed by `anyServes`, together with the corresponding theory change. Editing this model alone would otherwise make it violate `RefusalSaysWhy`.[1]

8–9. Cutoffs and host certification

No constant cutoff found. The hash extern also needs an agreement obligation tying its output to a specified canonical encoding and hashing procedure; injectivity requirements do not establish that computation.[1]

`10-world.md`

5. Against cited decisions; specification issues

```
before it is applied, it is tested
```

```
the protocol of agreement, with the program's own arguments
```

These describe two different temporal guarantees. The agreement protocol must execute the action to compare its effect. It can reject the result afterward, but cannot prevent the first disagreeing action from occurring. A destructive incorrect action has already acted before disagreement is detected. The chapter should distinguish prior validation against type laws from post-execution agreement evidence, rather than claim that this protocol supplies both.[1]

```
extern provisional is the declared exception
```

This sits alongside strict refusal of provisional cones in the same clause and in 6.S14. The chapter needs to state where the exception is allowed without reopening the strict-run decision.[1]

7. What I would change first

Correct the temporal claim: say which obligations are checked before execution and which are checked from the one executed action. This is consequential even after the token plumbing is repaired. [1]

8–9. Cutoffs and host certification

No constant cutoff found. The chapter's requirement that the actual action change only declared places still lacks a corresponding host-side law in `Host`; description-side preservation is insufficient. [1]

`06-certification.md`

1. Specification issue, separate from implementation findings

> `⊤ ... a failed check, refused by every cutoff`

A preorder with greatest element `⊤` permits the cutoff `⊤`, and reflexivity gives `⊤ ≤ ⊤`. The stated algebra therefore does not imply refusal by every cutoff. Either admissible cutoffs must exclude the failure element, or failed outcomes must be rejected independently of comparison. The Boolean draft chooses the latter through `OnlyAPassServes`; the chapter should make it explicit. [1]

5. Against cited decisions

> `whether they should be two cost types is left to the library`

D-2026-10-13-m already states two types and interfaces, `Distance` and `Evidence`. This sentence retains an older unresolved choice. D-2026-10-14-c postpones their implementation, not the distinction itself. [1]

> `An entry is kept per filling ... up to four`

"Up to four" is a literal retention-policy limit. If intended as normative, it conflicts with P11 and the kept-chain requirement. If it describes only the old implementation, label that limitation explicitly rather than carry it into the theory. [1]

7. What I would change first

Specify failed-outcome rejection independently of cost comparison. Otherwise the algebra's most permissive cutoff admits the very value called universally refused. [1]

8. Cutoffs

"Up to four" is the only explicit numeric policy limit I found in these six files. It is a retention limit, not one of the tolerance/confidence cutoffs listed in D-2026-10-14-c, but it still falls under the broader P11 requirement if normative. [1]

Not sure

- The actual behavior of `luthier.host.observe`, `print_line`, `read_file`, and `luthier.certify.cone_hash`: their Python bodies are not supplied.
- Whether the runner specially interprets the token-consuming Bool laws and returns the successor token and original result out of band. The comments claim this behavior; the definitions do not express it.
- Whether `Float64` admits NaN with ordinary comparison behavior. If so, `meet (Interval 0 1) (Interval NaN 2)` becomes `Interval 0 1` in the model, while the second input contains no numeric value; this breaks `TransportOnTheIntersection`. I cannot assert that case without the carrier's admitted values and comparison semantics.
- Whether external checking enforces certificate-constructor privacy, evidence/grade consistency, exact case domains, distribution compatibility, or complete dependency-cone enumeration. None is established by these six files.
- Parser acceptance is given by the prompt. Type checking, certification, and the actual runner behavior have not been verified here.[1]

⁂

1. A-prompt.md